CiteGuard — Panel Q&A; Bank

Possible judge questions with short answers. Team scodes · Track C · v0.4.0 · Live: citeguard-two.vercel.app

Tip: memorize the bold ideas; don't read paragraphs aloud. Point at CODEMAP.md and the Day 2 button.

Product & problem

Q. What is CiteGuard in one sentence?

A. It answers policy questions only from uploaded documents, with exact citations — or it refuses.

Q. Who is it for?

A. HR, compliance, and legal teams who cannot trust ChatGPT-style answers about company policy.

Q. What happens when the answer is not in the docs?

A. It says "I don't know," with zero citations. Refusal is a feature.

Q. Why is that better than a confident wrong answer?

A. Compliance needs a paper trail. A wrong fluent answer is more dangerous than a refusal.

Q. Track?

A. Track C — Knowledge & Compliance Agents.

Q. Live demo URL?

A. https://citeguard-two.vercel.app

Q. Repo?

A. https://github.com/Siddharthye/citeguard · team scodes · release v0.4.0

How it works

Q. Walk me through the ask path.

A. Upload/store → chunk → retrieve (current docs only) → answer → Citation Auditor → audit log.

Q. Do you always use an LLM?

A. No. Default is extractive (quote-based) so CI works without API keys. LLM is optional and still audited.

Q. How do citations work?

A. Each citation points to a document, chunk, and exact quote from that source.

Q. What is the Citation Auditor?

A. Runtime checks in faithfulness.ts: the quote must exist in the source, and the document must be currently effective.

Q. Is the auditor just a markdown prompt file?

A. No. agents/citation-auditor.md guides work, but the real gate is code + unit tests wired into /api/ask.

Q. Where should a judge start reading code?

A. CODEMAP.md, then policy-version.ts, faithfulness.ts, answer.ts.

Q. What is in the audit CSV?

A. Timestamp, question, refused flag, citation count, answer text — for a compliance trail.

Q. Why might audit CSV look empty on Vercel?

A. Serverless memory is per instance. Feature works; local/Docker is the reliable trail for demos.

Day 2 — superseded policies

Q. What was the Day 2 twist?

A. Policies can be superseded. Cite only the currently effective version; flag/refuse expired citations.

Q. How do you model versions?

A. Documents have effectiveDate, optional version, and policyFamily. Latest effective date on/before today wins in a family.

Q. What is policyFamily?

A. A group key (e.g. leave-policy). Same family = versions of one policy. Different families can still conflict.

Q. Conflict vs supersession?

A. Conflict: two current docs from different families disagree. Supersession: older same-family version is invalid for citing.

Q. In the Day 2 demo, what numbers should we see?

A. Default leave sample can show 18. Day 2 seed: old=12 (2020), current=22 (2024). Answer must be 22, not 12.

Q. What does the UI show?

A. Superseded banner naming the old doc; Sources badges current/superseded; citations from leave-policy-2024.md.

Q. What if retrieval somehow cites a superseded doc?

A. auditCitationCurrency fails the answer and we refuse rather than ship a stale cite.

Q. Did you rewrite the app for Day 2?

A. No. Additive: policy-version.ts + auditor currency check + upload metadata + demo endpoint.

Q. Where is the decision recorded?

A. DECISIONS.md ADR-007 and DAY2_PLAYBOOK.md.

Q. Why one-click Day 2 on the same API instance?

A. Vercel can split memory across instances; seed+ask in one request keeps the demo reliable.

Agent engineering & ADLC

Q. What is your constitution?

A. AGENTS.md + Spec Kit: groundedness over fluency; refuse over invent; CI without paid keys.

Q. Custom agent?

A. Citation Auditor — agents/citation-auditor.md, implemented in faithfulness.ts.

Q. Custom skill?

A. Chunk and Index — skills/chunk-and-index/SKILL.md, mirrored by chunk.ts + store.ts.

Q. What is Spec Kit for here?

A. Spec → plan → tasks trail under .specify/ so scope and ADLC are reviewable.

Q. How do agents “shape behavior”?

A. They are not only role cards: auditor rules become executable checks that can veto answers.

Architecture & trade-offs

Q. Why keyword retrieval instead of embeddings?

A. ADR-001: deterministic CI, no vector DB, auditable scores. Scoring boundary can later host embeddings.

Q. Isn't keyword retrieval weak?

A. For this demo corpus and Track C "cite or refuse," it is enough and testable. We prioritized correctness and CI.

Q. Chunking strategy?

A. Overlapping chunks with stable IDs so citations stay stable and retrieval has context overlap.

Q. PDF upload?

A. extract.ts turns uploads into plain text (txt/md/pdf path as implemented).

Q. Auth / multi-tenant?

A. Out of scope for the hackathon. Store is isolated in-memory/file; design leaves room for later auth.

Q. Why extractive default?

A. Keeps answers quote-faithful when LLM keys are missing; still optional LLM with auditor veto.

Q. Biggest technical risk?

A. Over-refusal (too strict) vs hallucination (too loose). We bias to refuse and tune thresholds with tests.

Q. What would you build next?

A. Persistent store, embeddings behind same score API, stronger PDF/DOCX, authenticated multi-tenant workspaces.

Testing, CI, delivery

Q. What does CI run?

A. Lint, unit tests, Playwright e2e (leave, pizza refuse, conflict, Day 2 supersession, CSV).

Q. Do tests need OpenAI keys?

A. No. Extractive path must pass without paid LLM keys.

Q. How do you prove Day 2?

A. Unit test cites 22 + notes superseded; Playwright one-click Day 2 demo.

Q. Docker?

A. Yes — docker compose up for a judge-local run.

Q. Release tag?

A. v0.4.0 — Day 2 supersession + CODEMAP + panel assets.

Q. Green Actions URL?

A. https://github.com/Siddharthye/citeguard/actions

Demo & panel logistics

Q. Exact 90-second demo?

A. Leave → 18 + open citation → pizza refuse → Day 2 button → 22 + badges. Then CODEMAP + ADR-007.

Q. Wi-Fi dies — plan B?

A. npm start or docker compose locally. Same try buttons.

Q. What not to open first on Vercel?

A. Don’t lead with audit CSV across cold instances. Lead with cite / refuse / Day 2.

Q. How long is the deck?

A. 10 slides — problem through live path. Speak from speaker notes; demo is the proof.

Q. Who on the team?

A. Team scodes. Assign: live clicker, ADLC speaker, Q&A.;

Hard / adversarial judge questions

Q. Isn’t this just RAG with extra steps?

A. RAG often still hallucinates. We force cite-or-refuse, runtime faithfulness, and currency — verified in CI.

Q. Can I jailbreak it into inventing policy?

A. Weak retrieval refuses. Auditor rejects ungrounded numbers/quotes. Adversarial unit tests cover nonsense queries.

Q. What if two current policies disagree?

A. multiSource / conflict banner — we surface disagreement instead of silently picking one.

Q. What if effectiveDate is in the future?

A. Currency resolution uses dates on or before today; future-dated docs are not treated as current yet.

Q. Why should we trust your demos aren’t scripted?

A. Live site + public repo + green CI + Playwright. Judges can click the same buttons.

Q. How is this different from “upload PDF to ChatGPT”?

A. ChatGPT can still invent and won’t reliably refuse, cite chunk-level quotes, or enforce superseded versions with tests.

Q. Where do you lose to a bigger team?

A. We traded flashy embeddings/UI for determinism and Day-2 readiness. Product is narrow by design.

Q. Show me the supersession code path.

A. filterCurrentChunks in answer.ts ← policy-version.ts; auditCitationCurrency in faithfulness.ts.